

SuperMoTo

The Super Monitoring Tool

Monitoring Matrix, Measurement & Speaker Correction

User and Technical Manual

Olivier Doaré, ENSTA
<https://perso.ensta.fr/~doare>
github.com/odoare
FX-Mechanics

Document version 0.1 — June 22, 2026

Contents

1	Overview	3
1.1	What SuperMoTo is	3
1.2	Why calibrate a monitoring system?	3
1.2.1	The room’s low-frequency modes	3
1.2.2	Boundary and early reflections	3
1.2.3	Imperfect loudspeakers	4
1.2.4	Subwoofers and the crossover region	4
1.2.5	Minimum phase, excess phase and invertibility	4
1.2.6	What SuperMoTo can and cannot do	5
1.2.7	Treat the room first	5
1.3	Signal flow at a glance	5
1.4	Plugin format and channel layout	6
2	Host (DAW) compatibility	7
2.1	Recommended and likely-compatible hosts	7
2.2	Hosts likely to be awkward	7
2.3	Suggested REAPER setup (reference)	7
3	The monitoring matrix	9
3.1	Concepts	9
3.1.1	What a frame contains	9
3.1.2	What an output (loudspeaker) contains	10
3.1.3	Configurations A–F	10
3.2	Interacting with the matrix	10
3.3	Per-output FIR correction and latency compensation	10
3.4	The global spectrum analyzer	11
3.5	The Configuration tool	11
3.5.1	Ambisonics decoding	12
3.6	Host-automatable parameters	13
3.7	Compact (collapsed) view	14
4	Measurement (Calibration view)	15
4.1	Running a measurement	15
4.2	The measurement microphone	16
4.2.1	Why a measurement microphone	16
4.2.2	Orientation matters	16
4.3	Microphone calibration	16
4.3.1	Why calibrate	17
4.3.2	The calibration file format	17
4.3.3	How SuperMoTo applies it	17
4.3.4	Practical recommendations	17

4.4	The three measurement modes	18
4.4.1	Dry — the raw loudspeaker	18
4.4.2	FIR — verifying the correction	19
4.4.3	System — the complete monitored chain	19
4.4.4	The “sent” channel convention	19
4.4.5	Capture timing	19
4.5	The SPL meter	19
5	Analysis & correction design	20
5.1	Transfer-function estimation (Welch method)	20
5.1.1	The H_1 estimator	21
5.1.2	Windowing, overlap and resolution	21
5.1.3	Propagation-delay estimation and removal	21
5.1.4	Complex spatial averaging	21
5.2	What the plot shows	22
5.3	Fractional-octave smoothing	22
5.3.1	Frequency-dependent smoothing	22
5.3.2	Why smoothing matters — and a recommendation	22
5.4	Designing the correction	23
5.5	Subwoofer phase alignment	25
5.5.1	Preserving the relative main/sub timing	25
5.5.2	The all-pass alignment term	25
5.5.3	Bulk time alignment: the <i>Mains delay</i> control	25
5.5.4	The complete correction filter	26
5.6	Rendering the FIR filter	27
5.6.1	Linear- / mixed-phase rendering	27
5.6.2	Minimum-phase rendering	27
5.7	Exporting	28
6	Workflows	29
6.1	A — Stereo monitoring with corrected speakers	29
6.2	B — 2.1 / bass-managed system with an aligned subwoofer	29
6.3	C — Verifying the complete system end to end	29
6.4	D — Multiple speaker sets / quick A/B	29
6.5	E — Ambisonics monitoring	30
7	Building from source	31
7.1	License	31
A	Glossary	32
B	Ambisonics primer	34
B.1	The idea: encode the field, decode to the rig	34
B.2	Spherical harmonics	34
B.3	Ambisonic order	34
B.4	The AmbiX convention: ACN and SN3D	35
B.5	Decoding	35
B.6	The $\max\text{-}r_E$ weighting	35

Chapter 1

Overview

1.1 What SuperMoTo is

SuperMoTo is an FX-Mechanics audio plugin (VST3, AU and Standalone) for the *monitoring section* of a studio. It is the big brother of [MoTo](#). From a single instance it lets you:

- route up to **32 inputs to 32 outputs** (8 by default) through a full routing matrix, with per-crosspoint gain, EQ, phase and delay;
- apply **one FIR correction filter per output** (zero-latency WDL convolution) to flatten each loudspeaker;
- **measure** the loudspeakers in the room with built-in stimuli;
- **analyse** those measurements and **design** the FIR correction curves (magnitude and phase), including phase-coherent **subwoofer integration**.

The plugin is organised in three functional parts that share one engine: the **Monitoring matrix** (Chapter 3), the **Measurement / Calibration** view (Chapter 4) and the **Analysis & correction design** view (Chapter 5).

1.2 Why calibrate a monitoring system?

What you hear at the listening position is never the loudspeaker alone: it is the loudspeaker *convolved with the room*, sampled by the microphone (or your ears) at one place. Several independent mechanisms each leave their mark on that combined response, and they behave very differently — which is exactly why some of them can be corrected electronically and some cannot.

1.2.1 The room’s low-frequency modes

Below roughly the **Schroeder frequency** (typically 150 Hz to 300 Hz in a domestic-sized room), the room behaves as a small set of discrete resonances — standing waves between parallel surfaces. Each **room mode** stores energy and releases it slowly, producing large peaks and dips in the magnitude response (often 10 dB or more) together with audible **ringing** (slow modal decay) in the time domain. Modes are strongly position dependent: a dip at one seat can be a peak half a metre away.

1.2.2 Boundary and early reflections

Sound radiated towards nearby surfaces returns to the listener delayed and attenuated. Reflections off the desk, floor, ceiling and side walls combine with the direct sound to produce **comb filtering** (regularly spaced peaks and notches) across the midrange and treble, and the speaker–boundary interference (**SBIR**) cancellation places a characteristic dip in the upper bass whose

frequency is set by the speaker-to-wall distance. Unlike modes these are essentially *wide-band* and tied to geometry.

1.2.3 Imperfect loudspeakers

Even in an anechoic chamber no speaker is perfectly flat: drivers have their own magnitude ripple and roll-offs, and the crossover between drivers adds **phase** shift, so the speaker is not minimum-phase (Section 1.2.5) everywhere. This part of the error is a property of the box, is the *same* at every listening position, and is the most cleanly correctable.

1.2.4 Subwoofers and the crossover region

Adding a subwoofer introduces a second source, in a different place, with its own roll-off and delay. Where the sub and the mains overlap their outputs add as *complex* pressures: unless they are in phase through the crossover the sum shows a dip or, worse, a deep cancellation [12]. Getting a smooth, coherent hand-off is as much a **phase / timing** problem as a level problem.

1.2.5 Minimum phase, excess phase and invertibility

Whether an error can be undone by a filter hinges on a single property of the response: whether it is **minimum-phase**.

Physically. A response is minimum-phase when, for its given magnitude, its energy arrives *as early as possible* — it has the least phase lag and the least *group delay* (the frequency-derivative of phase, i.e. the delay the response imposes near each frequency) consistent with that magnitude. Equivalently, both the response *and its inverse* are causal and stable, which is exactly what makes it correctable: undoing it does not require predicting the future. A single loudspeaker resonance, a driver roll-off and an isolated low-frequency room mode are essentially minimum-phase — the magnitude bump or dip and the phase rotation around it are two views of the *same* event. What is *not* minimum-phase: a pure propagation delay, a discrete reflection arriving after the direct sound, and the deep cancellation notch where two out-of-phase sources sum. These carry **excess phase** — lag beyond the minimum — that no stable causal filter can remove.

Mathematically. For a discrete linear time-invariant system with rational transfer function $H(z)$, minimum-phase means every pole *and* every zero lies strictly inside the unit circle. Poles inside make H causal and stable; zeros inside make the inverse $1/H(z)$ — whose poles are H 's zeros — causal and stable too, so H can be inverted by a realisable filter. Any stable response factors uniquely into a minimum-phase and an all-pass part,

$$H(z) = H_{\min}(z) H_{\text{ap}}(z), \quad |H_{\text{ap}}(e^{j\omega})| = 1, \quad (1.1)$$

where the all-pass factor H_{ap} (unit magnitude, so it changes only the phase) carries *all* the excess phase — each zero z_0 outside the unit circle reflected to $1/\bar{z}_0$ inside. The minimum-phase factor alone sets the magnitude, $|H| = |H_{\min}|$, and for it — and only it — magnitude and phase are not independent but a **Hilbert-transform pair**:

$$\arg H_{\min}(e^{j\omega}) = \mathcal{H}\{\log |H_{\min}(e^{j\omega})|\}, \quad (1.2)$$

so the phase is completely determined by the magnitude. SuperMoTo uses this identity both ways: to *design* a flat-phase correction (it conjugates the measured minimum-phase phase), and to *render* a low-latency filter from the magnitude alone (Section 5.6.2).

The practical consequence sets up the next section: SuperMoTo can invert the magnitude exactly and flatten the minimum-phase phase; the all-pass / excess part and any pure delay it

can only *pre*-compensate with a **linear-phase** filter (acausal, hence latency-adding, Section 5.6); and a genuine non-minimum-phase cancellation — a zero on or outside the unit circle — has no stable inverse at all, so boosting into it merely wastes headroom. This one distinction is why the spatial average, the boost ceiling and the frequency-dependent smoothing exist: they keep the design on the minimum-phase part of the response that is actually invertible.

1.2.6 What SuperMoTo can and cannot do

Correction is a linear filter applied to the signal *before* the speaker. That makes it powerful for some of the problems above and powerless for others:

- **Can do well.** Flatten the loudspeaker’s own magnitude and phase response; tame the broad *minimum-phase* part of the room’s low-frequency response that is common to the listening area; apply a defensible target/house curve; and **time- and phase-align a subwoofer** with the mains so they sum coherently through the crossover.
- **Cannot do.** It cannot remove energy that arrives *later* than the direct sound — a reflection is a separate event in time, so “correcting” for it only pre-distorts the direct sound and the dip usually returns elsewhere. Deep cancellation notches are *non-minimum-phase* and effectively non-invertible: boosting into them wastes headroom and excites the mode further rather than filling the null [9, 8]. And because the measurement is taken around one reference area, a correction is only valid *there* — it cannot be flat everywhere at once [11].

This is why SuperMoTo derives its correction from a **spatial average** of several microphone positions (so it follows the response that is consistent across the listening area rather than chasing one seat’s interference notches), limits its boost with a soft ceiling, and smooths the target more coarsely as frequency rises (Chapter 5).

1.2.7 Treat the room first

Electronic correction is the *last* step, not a substitute for acoustics. The errors it handles worst — reflections and modal ringing — are precisely the ones that respond to physical treatment, so the most effective gains come **upstream**, before any filter is designed:

- **Placement** of speakers and listening position relative to the walls (avoiding strong SBIR dips and the worst modal seats).
- **Symmetry** of the left/right setup, so the stereo image is not skewed by the room.
- **Bass trapping** in the corners to shorten modal decay — this reduces ringing, which no magnitude filter can.
- **Broadband absorption** at the first reflection points to reduce comb filtering.

A well-treated room makes the residual error smoother, more minimum-phase and more uniform across positions — exactly the kind of error a filter *can* fix — so room treatment and SuperMoTo are complementary, in that order.

1.3 Signal flow at a glance

Figure 1.1 shows the per-block processing performed by the matrix engine. Audio enters on the discrete input channels (8 by default, up to 32), is summed through the active matrix configuration(s), passes the per-output trim and FIR correction, is time-aligned across outputs, and finally the master section (Level / Mute / Dim / Mono) is applied before the discrete outputs. The active matrix size is set with the Inputs/Outputs selectors; the plugin bus supports 8, 16, 24 or 32 discrete channels.

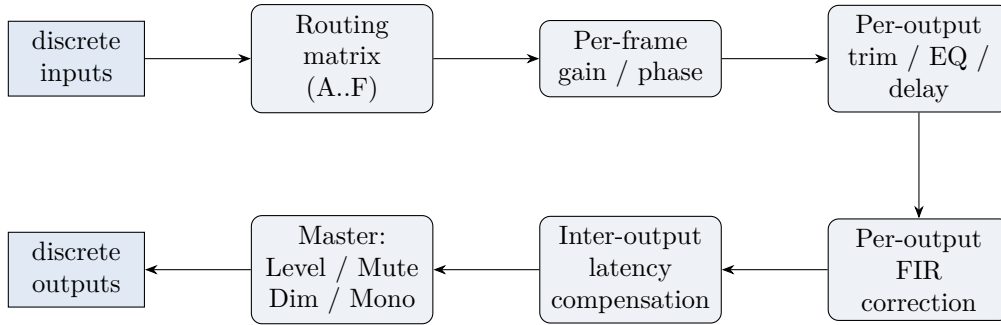


Figure 1.1: Per-block signal flow of the matrix engine. Each crosspoint (“frame”) of the matrix has its own gain, 4-band EQ, phase inversion and fractional delay; each output has a trim and an optional FIR correction filter. Outputs are then re-aligned so loudspeakers with different FIR lengths stay time-coherent.

1.4 Plugin format and channel layout

- Formats: **VST3**, **AU**, **Standalone**.
- Bus layout: one input bus and one output bus, each **16 discrete channels** (`AudioChannelSet::discrete`). The plugin only reports support for the exact 16-in / 16-out layout.
- Plugin type: audio effect (not a synth, no MIDI).

This fixed 16×16 discrete layout is the key constraint for host compatibility — see Chapter 2.

Chapter 2

Host (DAW) compatibility

SuperMoTo is primarily designed and tested in **REAPER**. To host it usefully a DAW must satisfy two requirements:

1. **Flexible channel routing** — the ability to feed the plugin’s discrete inputs from arbitrary sources and to send its discrete outputs to arbitrary hardware outputs / busses.
2. **A monitoring (control-room) insert point** — a place to put the plugin on the path between the mix and the physical speaker outputs, *after* the program material is mixed but *before* it leaves for the monitors, ideally not printed into renders/bounces.

2.1 Recommended and likely-compatible hosts

2.2 Hosts likely to be awkward

- **Pro Tools** — plugin channel formats are constrained to named surround layouts; an arbitrary high-count discrete insert is not the native model.
- **Logic Pro** — surround formats are fixed and discrete 16-channel routing through a monitor insert is not straightforward.
- **Studio One / FL Studio / Live** — usable as generic effect hosts, but they lack a true control-room monitor insert, so the plugin would have to live on the master bus and be manually bypassed for exports.

2.3 Suggested REAPER setup (reference)

1. Open View → Monitoring FX and insert SuperMoTo there so it processes the hardware output path and is excluded from renders.
2. In the plugin’s pin connector / channel mapping, map your mix busses to inputs 1..N and the plugin outputs 1..M to the physical speaker outputs.
3. Build your routing in the matrix (or with the Configuration tool) and store rigs in configurations A..F.

Host	Fit	Notes
REAPER	Primary	Per-track/track-channel routing up to 64 channels, and a dedicated <i>Monitoring FX</i> chain (View → Monitoring FX) that sits on the hardware output path and is excluded from renders. The reference environment.
Ardour	Strong	Arbitrary bus channel counts and flexible port routing; a dedicated <i>Monitor Section</i> on the master/monitor path. A natural fit on Linux.
Steinberg Nuendo / Cubase	Strong	The <i>Control Room</i> provides a monitor insert chain with flexible downmix and multi-channel busses — ideal for a post-production monitoring matrix. Confirm the Control Room insert accepts a 16-channel plugin layout.
Bitwig Studio	Possible	Very flexible routing and per-device chains; place the plugin on the master or on a hardware-output chain. No formal “monitor section”, so the monitoring insert must be built manually and bypassed before export.
Standalone (built in)	Always	The Standalone build talks straight to the audio device (up to 32 in / 32 out if the interface provides them); useful for a dedicated monitor-controller machine with no DAW.

Table 2.1: Hosts that can drive SuperMoTo. “Fit” is the author’s assessment based on the routing / monitor-section requirements; only REAPER is formally tested.

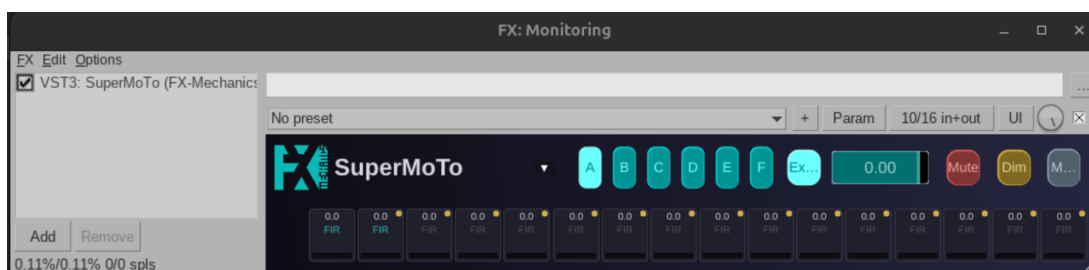


Figure 2.1: REAPER “Monitoring FX” window with SuperMoTo inserted, showing the 16-in/16-out pin/channel mapping to the hardware outputs.

Chapter 3

The monitoring matrix

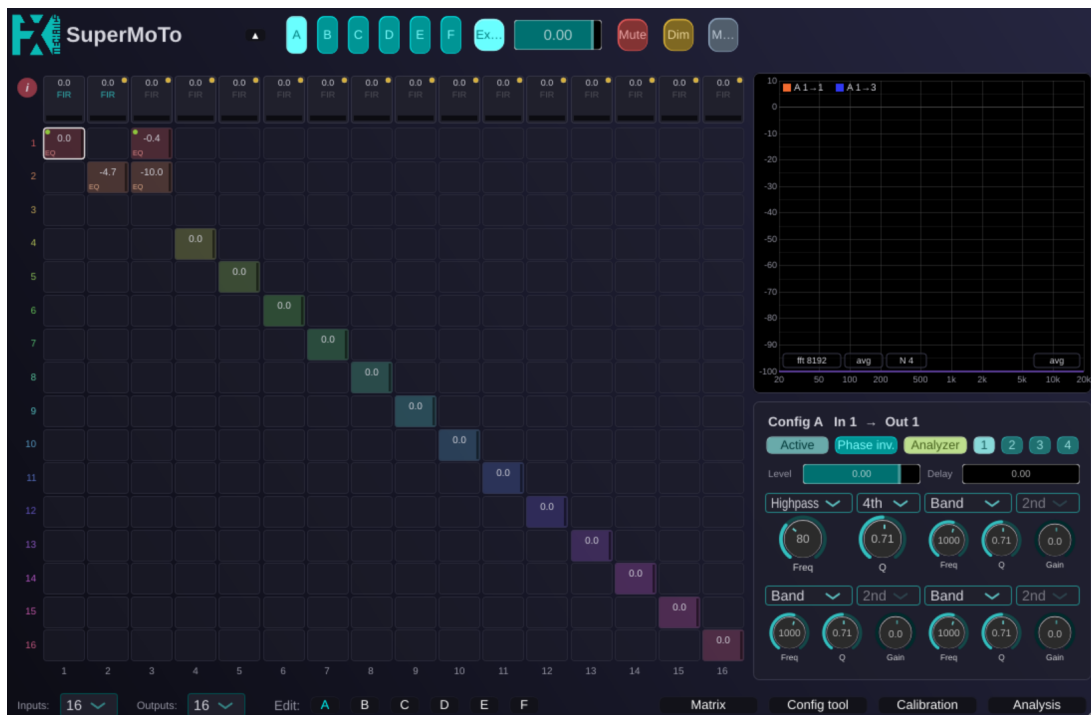


Figure 3.1: The Matrix view: top bar with configuration buttons A–F, Exclusive toggle, master Level and Mute/Dim/Mono; the 16×16 grid of frames each with its vu-meter; the per-output strip; the global spectrum analyzer; and the frame editor panel (bottom-right).

3.1 Concepts

The matrix is a grid of up to **32×32**. The cell at row *in*, column *out* is a **frame** (crosspoint): a pure *routing* cell that controls how much of input *in* is sent to output *out*. All the loudspeaker processing — trim, EQ, delay and the FIR correction — lives on the **output**, since it describes the physical speaker; output settings are global and therefore shared by all configurations.

3.1.1 What a frame contains

- **Active** flag (is this crosspoint routed at all);
- **Gain** (overall frame level, dB);

- **Phase invert** (a frequency-independent polarity flip, useful for e.g. a differential decode coefficient);
- a **vu-meter** and an **analyzer** checkbox to show the frame’s signal on the spectrum analyzer.

3.1.2 What an output (loudspeaker) contains

Each output is the chain applied to the summed signal of all active frames feeding it. Click an output cell (top strip) to edit it:

- **Trim** (output level, dB);
- a **4-band EQ** — each band can be lowpass, highpass, bandpass or peaking; lowpass/highpass are 2nd or 4th order; bands are cascaded in series. This is where the **bass-management crossover** lives (a highpass on a main, a lowpass on the sub);
- a **delay**, 0 ms to 100 ms, fractional (sub-sample), for **time-aligning** the speaker;
- one **FIR correction** filter (Section 5.6), loaded from a wav impulse response;
- a **vu-meter** and an **analyzer** checkbox.

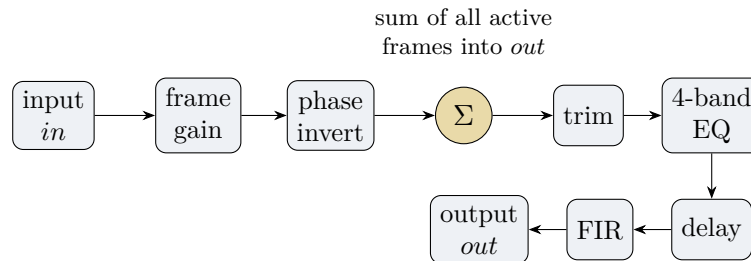


Figure 3.2: Routing frames (gain + phase) sum into each output, which then applies the per-speaker trim, EQ, delay and FIR correction.

3.1.3 Configurations A–F

There are **6 configurations** (A..F), each a complete 16×16 matrix, stored per preset. They are engaged from the top-bar buttons:

- **Exclusive** mode (default): engaging one releases the others — an instant A/B between rigs.
- **Non-exclusive** mode: active configurations are *summed*, so you can stack layers.

The configuration you are *editing* is chosen with the “Edit: A..F” buttons above the analyzer, independently of which one is engaged for listening.

3.2 Interacting with the matrix

3.3 Per-output FIR correction and latency compensation

Each output can carry one **FIR filter** (loaded from a wav impulse response, convolved with the zero-latency WDL engine) to compensate the attached loudspeaker. Because correction IRs are rendered with their energy *centred* (see Chapter 5), an FIR of length L adds about $L/2$ samples of delay to that output.

SuperMoTo applies **automatic inter-output latency compensation**: among the outputs fed by the engaged configuration, the ones with shorter (or no) FIR are delayed so that *all outputs stay time-aligned with the longest output FIR*. This is what keeps a subwoofer aligned with a linear-phase- corrected main. A **gold LED** on an output cell indicates it received compensation; hover to read the amount.

Gesture	Action
Click a frame	Select it (opens it in the frame editor panel).
Double-click a frame	Activate / deactivate the crosspoint.
Vertical drag	Adjust the frame gain.
Alt+click	Toggle the frame's analyzer trace.
Right-click	Context menu for the frame.
Click an output	Select it (opens it in the output editor: trim, EQ, delay, FIR).
Output: double-click	Toggle that output's FIR on/off.
Output: vertical drag	Adjust the output trim.
Output: alt+click	Toggle the output-sum analyzer trace.
Output: right-click	Load / clear the impulse-response (FIR) wav file.

Table 3.1: Mouse interactions in the matrix and output strip. Every page also has an info button (i) summarising its controls and shortcuts.



Figure 3.3: Output strip close-up: the gold LED marks an output that received automatic latency compensation; the tooltip shows how many samples / milliseconds were added to keep it aligned with the longest FIR.

3.4 The global spectrum analyzer

The analyzer can display any combination of:

- individual matrix **frames** (alt+click / frame checkbox), up to 8 simultaneous frame traces;
- **output sums** (output-strip checkbox).

A clickable **avg** / **peak** badge switches the per-point aggregation. The analyzer also has buttons to change the vertical-axis limits, and zooming is driven by mouse interaction on the plot.

3.5 The Configuration tool

For standard layouts the Configuration tool writes a whole matrix for you. Choose a layout (**2.0**, **2.1**, **4.0**, **4.1**, **5.1**, **7.1**), assign the input and output channel of each speaker, set the gains and the bass-management crossover frequency, then **Apply** to a configuration (A..F). The mains are high-passed and their sum is sent, low-passed, to the subwoofer output. The result is an ordinary configuration you can fine-tune by hand afterwards.

3.5.1 Ambisonics decoding

The tool also generates **periphonic (3D) Ambisonics** decoders for orders 1–3 (Appendix B is a primer on the underlying theory). It follows the **AmbiX** convention — channels in **ACN** order with **SN3D** normalisation — and takes the B-format signal on the first $(\text{order} + 1)^2$ inputs (4, 9 or 16 channels). Instead of one input per speaker, each row sets a loudspeaker’s **azimuth**, **elevation** and **radius** (distance from the centre) and its output channel. The **Speakers** control chooses the number of outputs: at the canonical count the tool ships a hand-tuned periphonic rig (an 8-speaker cube, a 12-speaker icosahedron, a 16-speaker dome); at any other count it lays the speakers out on a near-uniform spherical-Fibonacci lattice. Either way you then edit the angles and radii to match the real installation (the minimum is $(\text{order} + 1)^2$ speakers, the number of harmonics).

Apply builds a **max-rE sampling decoder**: for each speaker s in direction Ω_s and each harmonic j the decode gain is

$$D[s][j] \propto a_{n(j)} Y_j^{\text{SN3D}}(\Omega_s), \quad (3.1)$$

where Y_j^{SN3D} is the real SN3D spherical harmonic of ACN index j , a_n the per-order max-rE gains (from the largest root of the Legendre polynomial $P_{\text{order}+1}$, improving localisation), and the matrix is scaled so an on-axis plane wave decodes to about unity. Each non-negligible $D[s][j]$ is written as a harmonic $\rightarrow$ speaker frame, its sign becoming a polarity flip. With bass management enabled, the omnidirectional W component (ACN 0) is low-passed to the subwoofer while the speaker feeds are high-passed — which, summed at each output, is exactly the high-passed decoded speaker because the filters are linear. The matrix grows to expose the B-format inputs and the speaker (plus sub) outputs. Refine gains, delays and per-output FIR corrections in the matrix afterwards.

Importing an IEM AllRADecoder. As an alternative to the built-in sampling decoder, **Load IEM decoder...** reads a `.json` configuration exported by the IEM *AllRADecoder* (part of the IEM Plug-in Suite). AllRAD (all-round ambisonic decoding) handles irregular and hemispherical rigs better than a plain sampling decoder. The file’s decode **Matrix** replaces the computed one; SuperMoTo converts it to its SN3D / ACN feed on the fly:

- if **ExpectedInputNormalization** is `n3d`, each coefficient of ACN degree n is multiplied by $\sqrt{2n+1}$ ($\text{N3D} \rightarrow \text{SN3D}$);
- if **Weights** is `maxrE` and **WeightsAlreadyApplied** is `false`, the per-degree max-rE gain is folded in;
- **Routing** maps each matrix row to a hardware output channel; **IsImaginary** loudspeakers (used only to regularise the decoder design) carry no row and are skipped;
- the loudspeaker layout supplies each speaker’s azimuth / elevation, its **Radius** (used by the radius compensation below) and its per-speaker **Gain** (folded into the frame gain as a trim).

A gain matrix cannot encode time, so the imported decoder — exactly like the built-in one — carries *no* delay: the radius delay below is the only time alignment, and it is optional.

Radius compensation. Ambisonic decoding assumes the speakers are equidistant from the centre. When they are not, the tool corrects for each **radius** r_s relative to the *farthest* speaker r_{max} , writing a per-speaker output **delay** and **trim**:

$$\Delta t_s = \frac{r_{\text{max}} - r_s}{c}, \quad \Delta g_s = 20 \log_{10} \frac{r_s}{r_{\text{max}}} \ (\leq 0), \quad (3.2)$$

with $c = 343 \text{ m s}^{-1}$. Closer speakers are thus delayed (so all wavefronts reach the centre together) and attenuated (inverse-distance law); the reference being the farthest speaker, no channel is ever

boosted or asked for a negative delay. Each part is independently optional — **Write radius gain** and **Write radius delay** — and both apply equally to the built-in and the imported (IEM) decoders; turning one off leaves that output property untouched, so an alignment or level already set (e.g. in the analysis pane) survives an Apply. The radius delay is independent of, and rides on top of, the inter-output FIR latency compensation (Section 3.3), which only equalises the FIR processing latency: the final arrival of a speaker is $\Delta t_s + l_{\max}$, so the radius alignment survives untouched.

Measured vs. geometric alignment (do not double-count). The geometric radius delay above and the *measured* subwoofer time-alignment (Section 5.5.3) are **two routes to the same delay**, not two delays to add. When you run the per-output sub analysis for a main speaker k at distance d_k from the mic centroid, the main set has d_k/c removed per file and the sub is anchored on that same delay, so the reported “Mains delay” is $\approx (d_{\text{sub}} - d_k)/c$ plus the intrinsic group delay. Setting that on the output gives an acoustic-plus-electronic arrival of

$$\underbrace{d_k/c}_{\text{acoustic}} + \underbrace{(d_{\text{sub}} - d_k)/c}_{\text{electronic}} = d_{\text{sub}}/c, \quad (3.3)$$

independent of d_k : every measured main lands at the same time as the sub *and* as the other mains. So a per-speaker measured alignment already contains the path-length (radius) term — and more accurately than an entered radius, since it uses the true acoustic path, references the same centroid as the average, and folds in the speakers’ intrinsic group delay. Therefore:

- **Measured route** (best): run the sub analysis per main and set each output delay to the reported value. To (re)build the decode without clobbering those delays, turn **Write radius delay** off before Apply — the tool then writes the decode, the crossover and the radius *trim* (unless **Write radius gain** is also off) but leaves each output’s delay untouched. (Otherwise set the radii equal so the geometric delay is zero.)
- **Geometric route** (quick): enter the radii and let the tool write the delay and trim; use this when there is no sub, or you have not measured each speaker.

One thing the measured route does *not* give you is level matching: the correction normalises each speaker to its own mid-band, so inter-speaker **amplitude** differences are not equalised. The radius **trim** Δg_s (or a calibrated-SPL measurement) still handles that.

3.6 Host-automatable parameters

The matrix settings themselves are deliberately *not* exposed as host parameters: $6 \times 32 \times 32$ frames with ~ 10 fields each would swamp any host. They are saved with the plugin state. Only the master controls are real host parameters:

Parameter	Type / range	Meaning
Level	float, -60 to 6 dB	Master output level.
A...F	bool (6)	Engage configuration A–F (default: A on).
Exclusive	bool (default on)	Engaging one config releases the others; off = sum.
Mute	bool	Mute master output.
Dim	bool	Dim master by 6 dB (factor 0.5).
Mono	bool	Mono sum.

Table 3.2: The complete set of host-automatable parameters (APVTS).

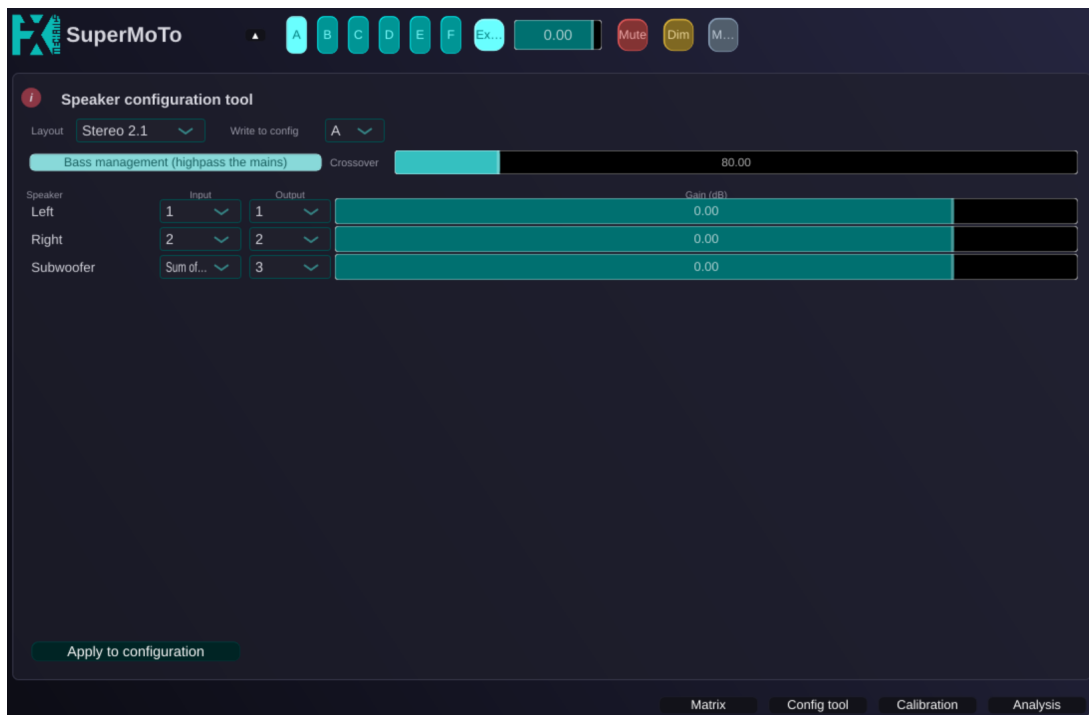


Figure 3.4: The Configuration tool: a standard layout (here 5.1) with per-speaker input/output assignment, gains and the bass-management crossover frequency, ready to Apply into one of the A–F configurations.

3.7 Compact (collapsed) view

The collapse button (▲) shrinks the window to a compact *output-strip-only* view, handy as a permanent monitor controller once the routing is set.

Chapter 4

Measurement (Calibration view)

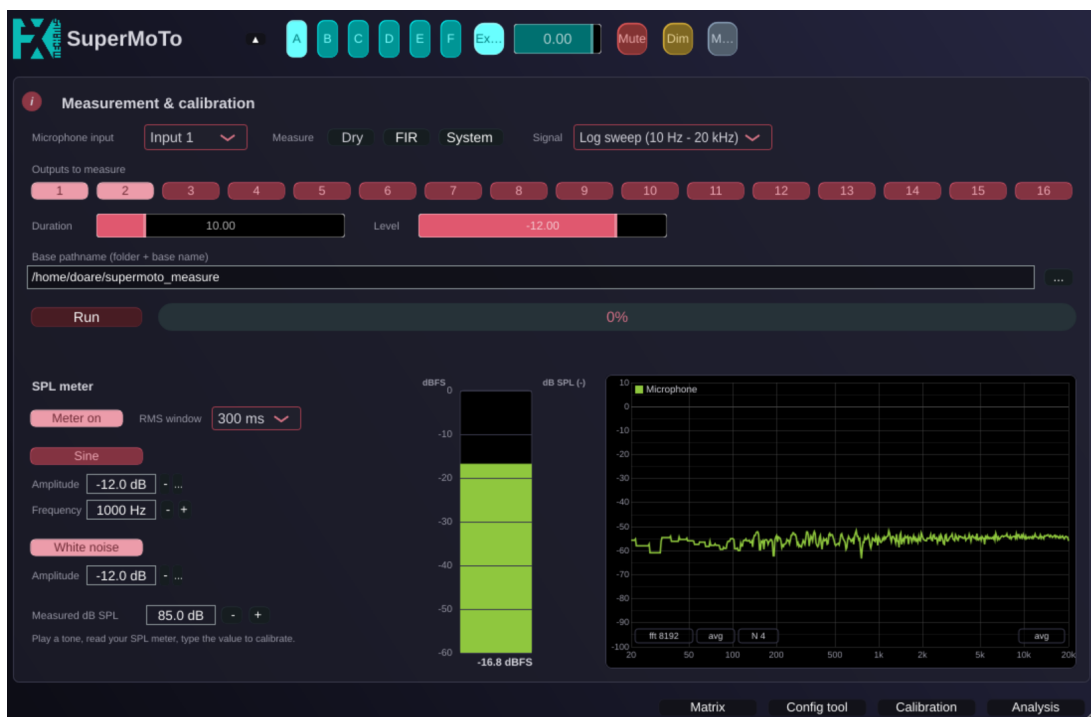


Figure 4.1: The Calibration view: microphone input, measurement type, channel selection, stimulus (noise / sweep), duration, level and base path, the Run control, and the right-hand SPL-meter section with its dual dBFS / dB SPL bar and test-signal generators.

4.1 Running a measurement

Select the **microphone input**, the **measurement type**, the **channels** to measure, the **stimulus** (band-limited white noise or a logarithmic sweep, 10 Hz to 20 000 Hz), the **duration** (5 s to 30 s), the **level** and the **base pathname**. **Run** measures each selected channel in turn and writes a stereo wav file per channel:

- channel 1 = the **sent** signal (the raw stimulus),
- channel 2 = the **recorded** microphone signal.

4.2 The measurement microphone

What the analysis sees is the loudspeaker and room *as sampled by the microphone*. The microphone is therefore part of the measurement chain, and the kind of microphone — and how well it is characterised — sets a ceiling on how trustworthy the resulting correction can be.

4.2.1 Why a measurement microphone

Ordinary recording microphones (cardioid vocal or instrument mics) are deliberately *voiced*: they have a directional pickup pattern and a shaped, non-flat response designed to flatter sources. That is exactly wrong for measurement, where you want the microphone to report the sound field as neutrally as possible. A **measurement microphone** is built for this:

- **Omnidirectional**, so it captures the room as the ear does rather than rejecting it from the sides.
- **Small diaphragm** (typically 6 mm, quarter-inch or half-inch), for a wide, uniform high-frequency response with little directional colouration.
- **Nominally flat** magnitude response over the audio band, with only small residual deviations — which a calibration file then removes (Section 4.3).

Common, affordable choices include the USB miniDSP **UMIK-1** / **UMIK-2**, the Dayton Audio **EMM-6** (XLR) and **iMM-6**, the Behringer **ECM8000**, and any class 1/class 2 measurement capsule. USB models carry their own ADC and appear as a soundcard input; XLR models need phantom power and an audio interface. Either works — SuperMoTo only needs the microphone to arrive on one of its plugin input channels.

4.2.2 Orientation matters

Because the microphone is omnidirectional only at low frequencies, its high-frequency response depends on the angle of arrival, so it is characterised for a specific **orientation**:

- **0° (on-axis)** — capsule pointed *at the loudspeaker*. This is the orientation for the swept-speaker measurements SuperMoTo makes, and the one whose calibration file you should load.
- **90° (grazing)** — capsule pointed at the ceiling, used for steady-state room / RTA averaging where sound arrives from many directions.

Vendors such as miniDSP ship *two* calibration files for this reason; use the 0° file and point the microphone at the speaker under test.

4.3 Microphone calibration

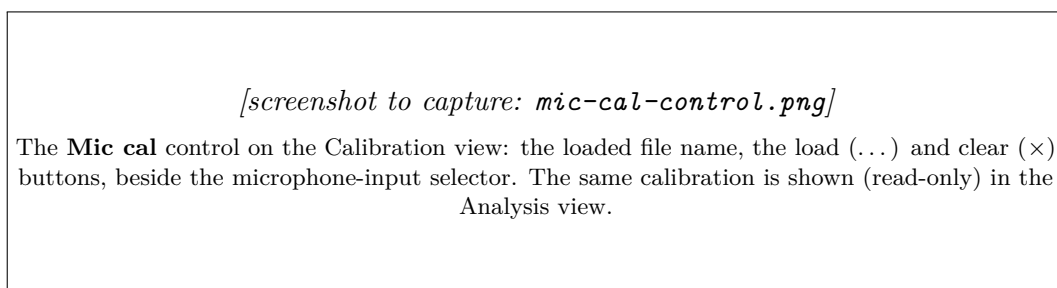


Figure 4.2: The **Mic cal** control on the Calibration view: the loaded file name, the load (...) and clear (x) buttons, beside the microphone-input selector. The same calibration is shown (read-only) in the Analysis view.

4.3.1 Why calibrate

Even a good measurement microphone is not perfectly flat: each capsule has small deviations from flat, most noticeably a gentle rise or roll-off in the top octave (above 8 kHz to 10 kHz) and sometimes a few tenths of a dB elsewhere. The **calibration file** is the measured deviation of that microphone from flat. If it is not removed, the analysis attributes the microphone’s own colouration to the loudspeaker and *builds it into the correction* — for example, dipping the speaker’s treble to compensate a rise that exists only in the microphone. Dividing the calibration out first means the estimated transfer function reflects the **loudspeaker and room alone**.

4.3.2 The calibration file format

There is no formal standard, but a single de-facto plain-text format is shared by REW, miniDSP, Dayton (OmniMic / iMM-6) and the FRD tools, and SuperMoTo reads it directly (`.txt`, `.cal` or `.frd`):

- comment lines start with `*`, `#` or `;` (the miniDSP `Sens Factor =...` header is one such line);
- every other line is `frequency_Hz` `magnitude_dB` [`phase_deg`], whitespace- or comma-separated, with the phase column optional.

The magnitude is the microphone’s deviation from flat (mean ≈ 0 in the midband). Where do the numbers come from? **Individually-measured** mics (e.g. the UMIK series) have a file keyed to the unit’s serial number, downloaded from the vendor; cheaper mics may ship only a *model-typical* file, or none. The `Sens Factor` line encodes absolute sensitivity for dB SPL work and is ignored here — only the frequency-response curve is used.

4.3.3 How SuperMoTo applies it

Because there is only one physical microphone, the calibration is a **single global setting**: load it once with `Mic cal ...` on the Calibration view (it sits next to the microphone-input selector), and it is remembered across sessions and projects. The Analysis view shows the same calibration read-only (*Mic cal: ...*) as a reminder that it is in effect. It is used in two places:

1. **The live spectrum display** (SPL-meter section) plots the *corrected* microphone spectrum, so what you see already has the microphone’s own response taken out.
2. **The analysis** divides the microphone response out of every measured transfer function before the average, the correction and the exported curves are computed.

Formally, at each frequency bin the measured response is multiplied by the inverse of the microphone curve,

$$H_{\text{corr}}(f) = H_{\text{meas}}(f) 10^{-c_{\text{dB}}(f)/20} e^{-j c_{\phi}(f)}, \quad (4.1)$$

where c_{dB} and c_{ϕ} are the calibration magnitude and phase, interpolated in log-frequency and held flat (at the endpoint value) outside the file’s range. When the file has no phase column, $c_{\phi} \equiv 0$ and only the magnitude is corrected.

Two consequences are worth stressing:

- **The recorded measurement files stay raw.** The calibration is applied at *analysis* time, not baked into the captured wavs, so you can change or replace the calibration (a better file, the other orientation) and simply re-analyse — no need to re-measure.
- **The broadband SPL number is not corrected.** A single dB SPL reading cannot carry a frequency-dependent curve; only the spectrum display and the analysis use the calibration. (Absolute dB SPL is set separately, Section 4.)

4.3.4 Practical recommendations

- Load the 0° calibration file and point the microphone *at the speaker* for the swept measurements.

- A calibration matters most **above a few kHz**, where the microphone’s own deviations live and where room averaging no longer hides them; in the bass the capsule is essentially flat and room modes dominate regardless.
- Loading *no* calibration is acceptable for broad bass correction, but then any treble target inherits the microphone’s error — so for full-range correction a calibrated microphone is strongly advised.
- The calibration corrects the microphone, *not* its placement: mic position and spatial averaging (Section 5.1.4) remain just as important.

4.4 The three measurement modes

The **measurement type** selector decides *where* the stimulus is injected and *how much* of the engine it passes through before reaching the loudspeaker and the microphone. The three modes form a natural progression: characterise a bare speaker (**Dry**), verify its correction (**FIR**), then validate the whole monitored chain (**System**). Table 4.1 summarises them and Figure 4.3 shows the injection point of each.

Mode	What is exercised	File name
Dry	Stimulus written straight onto the measured <i>output</i> — no trim, no FIR. The raw loudspeaker, as wired. The first step before designing a FIR.	<base>_<output>.wav
FIR	Stimulus to the measured <i>output</i> through its trim and FIR chain only, to verify a correction by re-measuring.	<base>_<output>.wav
System	Stimulus injected into a plugin <i>input</i> and run through the whole engine (matrix routing, frame EQ / crossover, output FIRs, inter-output latency compensation) — the complete system as heard. Channel selection means <i>inputs</i> in this mode.	<base>_in<input>.wav

Table 4.1: Measurement modes. “Dry” characterises a bare speaker, “FIR” verifies its correction, “System” captures the complete monitored chain.

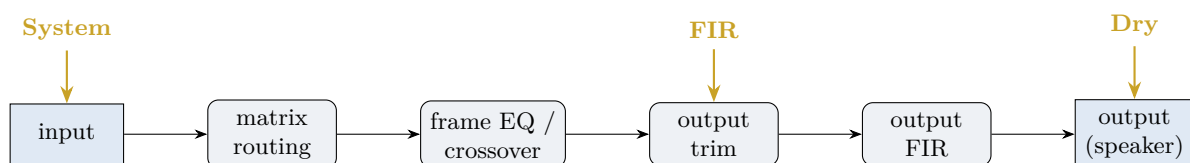


Figure 4.3: Where each mode injects the stimulus. **Dry** drives the output directly (bypassing trim and FIR); **FIR** enters at the output trim so the signal passes trim + FIR; **System** enters at a plugin input and traverses the entire engine.

4.4.1 Dry — the raw loudspeaker

The stimulus is written directly onto the selected output channel; the output trim and FIR are *not* applied. The capture therefore characterises the loudspeaker and room exactly as the speaker is physically driven, with no correction in the path. This is the measurement you feed to the analysis part (Chapter 5) to *design* a correction. Run it for each output you intend to correct, at several microphone positions around the listening spot.

4.4.2 FIR — verifying the correction

The stimulus is emitted to the selected output through that output’s **trim and FIR** only (the matrix is otherwise silent). Use it *after* you have exported a correction IR and assigned it to the output: re-measuring should show a flatter response than the corresponding Dry capture. It is the direct before/after check on a single speaker’s correction.

4.4.3 System — the complete monitored chain

The stimulus is injected into the selected plugin *input* and run through the whole engine with the currently engaged configuration(s): matrix routing, frame EQ and bass-management crossover, the output FIRs and the inter-output latency compensation — exactly what you hear. The master gain is bypassed (unity) during the run, so the capture level is set by the stimulus alone. Because the source is an input, **the channel selection chooses inputs** and the files are tagged *_in* so a System set never collides with a Dry/FIR (output) set. This is the end-to-end validation: for a bass-managed rig it shows whether mains and subwoofer sum smoothly through the crossover at the listening position.

4.4.4 The “sent” channel convention

In *every* mode the captured stereo file stores the **raw stimulus** (pre-everything) on channel 1 and the microphone on channel 2. The transfer function computed in the analysis part is therefore always *microphone / raw stimulus*. The consequence is deliberate and important:

- in **Dry** mode that ratio is the bare speaker + room response;
- in **FIR** and **System** modes it already *includes* the correction (and, for System, the full chain), so a successful correction shows up directly as a flatter measured transfer function — you do not have to subtract anything by hand.

4.4.5 Capture timing

Each capture records the stimulus plus a short decay **tail** (1 s) so the room’s late energy is included, and the stimulus is faded in and out (20 ms) to avoid clicks. When measuring several channels in one run, the engine holds the monitor path silent while each finished capture is written to disk, so the speakers are not blasted between consecutive measurements.

4.5 The SPL meter

The right-hand SPL-meter section reuses the microphone / channel / measurement-type selections. It shows the microphone RMS on a dual **dBFS / dB SPL** bar and can emit a sine and/or white-noise test signal, routed exactly like the selected measurement mode. To calibrate the dB SPL scale: play a tone, read a real SPL meter at the listening position, and type that value into the plugin.

Chapter 5

Analysis & correction design



Figure 5.1: The Analysis view: individual smoothed transfer functions (thin), their average (thick), the proposed correction and the corrected response, in magnitude and phase; with the n th-octave smoothing, correction-level, boost-ceiling and analysis-range controls, the subwoofer integration section (crossover, Invert, Mains delay), and the Export buttons.

5.1 Transfer-function estimation (Welch method)

Load the set of measurement files of *one* speaker, the microphone moved to several positions around the reference listening point. For each file the plugin estimates the loudspeaker-room transfer function $H(f) = Y(f)/X(f)$, where x is the emitted stimulus (channel 1 of the file) and y the captured microphone signal (channel 2). If a microphone calibration is loaded, it is divided out of H at this stage (Section 4.3, eq. (4.1)), so everything below describes the loudspeaker and room rather than the microphone.

5.1.1 The H_1 estimator

A direct bin-by-bin division Y/X is unusable in a room: it is dominated by measurement noise wherever X is small, and a single FFT gives an estimate with 100 % standard error. SuperMoTo instead uses the classical **Welch averaged-periodogram** approach [1]: the recording is cut into overlapping segments, each is windowed and transformed, and the *auto*- and *cross-spectra* are averaged over the L segments,

$$\hat{P}_{xx}(f) = \frac{1}{L} \sum_{\ell=1}^L |X_{\ell}(f)|^2, \quad \hat{P}_{xy}(f) = \frac{1}{L} \sum_{\ell=1}^L X_{\ell}^*(f) Y_{\ell}(f). \quad (5.1)$$

The transfer function is then formed as the H_1 **estimator**

$$\hat{H}(f) = \frac{\hat{P}_{xy}(f)}{\hat{P}_{xx}(f)}. \quad (5.2)$$

H_1 is the least-squares estimator that is optimal when the noise is on the *output* (the microphone path) — the realistic case for an acoustic measurement — and it rejects that noise as more segments are averaged. (Its companion $H_2 = P_{yy}/P_{yx}$ assumes input noise; both bracket the true response, and they coincide only when the coherence is unity.) The derivation and noise properties of H_1/H_2 are standard material in [2, 3, 4].

5.1.2 Windowing, overlap and resolution

Each segment is weighted by a **Hann window** before the FFT, to control spectral leakage [5]; consecutive segments overlap by **50 %**, the standard choice that makes the Hann-windowed segments use the recording almost uniformly (Welch’s WOSA scheme). The window length W is user-selectable (4096 to 262 144, default $W = 65536$) and sets the frequency resolution

$$\Delta f = \frac{f_s}{W}, \quad (5.3)$$

e.g. ≈ 0.67 Hz at $f_s = 44.1$ kHz with the default window. A long window resolves low-frequency room modes; the averaging over segments (and later over microphone positions) is what buys the variance reduction. Bins where $\hat{P}_{xx}$ is essentially zero are regularised by a small term $\varepsilon = 10^{-10} \max_k \hat{P}_{xx} + 10^{-30}$ added to the denominator of (5.2), so silent bins do not blow up.

5.1.3 Propagation-delay estimation and removal

Each microphone position is at a different distance from the speaker, so each $\hat{H}$ carries a different bulk propagation delay — a linear phase term $e^{-j2\pi f\tau}$. Averaging the complex responses *without* removing it would let the position-dependent phase ramps cancel each other and destroy the result. The plugin therefore estimates the delay τ of each measurement from its impulse response (inverse FFT of $\hat{H}$), taken as the position of the peak of $|h(t)|$ (a peak in the second half of the buffer is interpreted as a small negative delay that wrapped around). The linear phase is then removed,

$$\hat{H}'(f) = \hat{H}(f) e^{+j2\pi f\tau}, \quad (5.4)$$

which time-shifts every measurement so its main arrival sits at $t = 0$. This is the excess-delay normalisation used in swept-sine room measurement practice [6].

5.1.4 Complex spatial averaging

The delay-aligned responses of all N positions are then averaged as *complex* quantities,

$$\bar{H}(f) = \frac{1}{N} \sum_{n=1}^N \hat{H}'_n(f). \quad (5.5)$$

Complex (vector) averaging preserves phase and, more importantly, averages out the position-dependent comb filtering caused by reflections: narrow peaks and nulls that move with the microphone tend to cancel, leaving the response the loudspeaker reproduces *consistently* across the listening area. This spatial averaging over a set of positions around the seat is the approach advocated for room/loudspeaker correction in [11, 8]; correcting a single-point measurement, by contrast, tends to “correct” room nulls that do not exist elsewhere.

5.2 What the plot shows

- the **individual** smoothed transfer functions $\hat{H}'_n$ (thin lines);
- their complex **average** $\bar{H}$ (thick line);
- the proposed **correction** C ;
- the **corrected response** $\bar{H}C$.

Magnitudes are normalised to the 200 Hz to 2000 Hz mean of the average (so 0 dB is the mid-band level, independent of the absolute measurement gain); phase is shown as principal value with the propagation delay already removed.

5.3 Fractional-octave smoothing

A $1/n$ -**octave smoothing** is applied both to the displayed curves and to the average from which the correction is derived. It is a **complex** moving average — magnitude and phase together — over a logarithmic band $[k/r, kr]$ around each bin k , with $r = 2^{\text{fraction}/2}$, evaluated in $O(n)$ with a prefix sum. Working on a constant- Q (fractional-octave) grid matches the ear’s roughly logarithmic frequency resolution and prevents the correction from chasing the dense, narrow interference structure of the room; the rationale and the complex-smoothing formulation follow Hatziantoniou and Mourjopoulos [7]. The available settings are **Off**, **1/24**, **1/12**, **1/6**, **1/3**, **1/2** and **1 octave** (the 1/2-octave step fills the large perceptual gap between 1/3 and 1 octave).

5.3.1 Frequency-dependent smoothing

The smoothing fraction is **not** a single global value: a *low*-frequency fraction and a *high*-frequency fraction are set independently (the **Smooth LF/HF** controls), and the octave fraction used at bin k is interpolated log-linearly in frequency between them,

$$\text{frac}(f) = \text{frac}_{\text{LF}} + t(f) (\text{frac}_{\text{HF}} - \text{frac}_{\text{LF}}), \quad t(f) = \text{clamp}\left(\frac{\log_2 f - \log_2 f_{\text{lo}}}{\log_2 f_{\text{hi}} - \log_2 f_{\text{lo}}}, 0, 1\right), \quad (5.6)$$

with anchors $f_{\text{lo}} = 100$ Hz and $f_{\text{hi}} = 10$ kHz: at and below 100 Hz the LF fraction is used, at and above 10 kHz the HF fraction, and the two are blended in between. Setting both controls to the same value gives uniform smoothing (the classic behaviour). This makes the low-vs-high strategy below something you set *directly*, in a single correction, rather than having to approximate it. Note that this is distinct from the constant- Q widening that any fractional-octave window already has: that widens the window in Hz as frequency rises but keeps the *octave* fraction constant; here the octave fraction itself varies with frequency.

5.3.2 Why smoothing matters — and a recommendation

Do not try to correct a raw or lightly-smoothed response. An unsmoothed in-room measurement is full of sharp, deep nulls and peaks produced by reflections and standing waves. These narrow features have three properties that make inverting them counter-productive:

- they are largely **non-minimum-phase** — true cancellations that *cannot* be undone by any magnitude/phase filter; attempting it only demands enormous boosts that the soft-knee ceiling (Section 5.4) then has to claw back, wasting headroom [9, 8];

- they are **position-dependent** — a null at the measurement point has moved or vanished a few centimetres away, so “filling” it corrects one point while *degrading* every other listening position;
- inverting a narrow notch produces a long, high- Q filter whose **ringing** (pre-/post-echo in the rendered FIR) is itself audible.

The correction here is derived from the *smoothed* complex average, so the smoothing setting is your primary control over how much of this fine structure the inverse is even allowed to see. Combined with the complex spatial averaging of Section 5.1 (which already cancels much of the position-dependent comb filtering), enough smoothing keeps the correction on the **broad, reproducible trends** of the loudspeaker–room response — exactly the part that is stable across the listening area and safe to invert.

Practical guidance.

- Treat the corrected curve, not the raw one, as the target: it should look **smooth**, not a mirror image of every wiggle in the measurement. If the correction curve is spiky, increase the smoothing.
- Exploit the frequency-dependent smoothing (Section 5.3.1): it is generally safe to correct more aggressively at **low frequencies** (broad, modal, relatively position-stable) and only lightly at **high frequencies**, where the response is dominated by direct sound plus early reflections and narrow dips should be left alone — broad trends only [11]. A good starting point is a *finer* LF fraction (e.g. 1/12–1/6 octave) with a *coarser* HF fraction (e.g. 1/3–1 octave).
- For uniform smoothing, set both controls equal; **1/6 octave** on both is a sensible neutral baseline. If a correction sounds harsh, “phasey” or over-processed, move *coarser* and/or lower the **correction level** (Section 5.4) rather than fighting individual peaks.
- Use the **analysis range** to keep the correction off the extreme top and bottom entirely, where even heavy smoothing cannot make the data trustworthy.
- Smoothing is your first line of regularisation; the boost ceiling and analysis-band fade are the safety nets, not a substitute for choosing a sensible smoothing.

5.4 Designing the correction

The correction $C(f)$ compensates **both modulus and phase**. Starting from the normalised smoothed average $h = \bar{H}_s/g_{\text{ref}}$ (with g_{ref} the 200 Hz to 2000 Hz mean magnitude), the ideal inverse is the complex reciprocal

$$C_0(f) = \frac{1}{h(f)} = \frac{\overline{h(f)}}{|h(f)|^2}, \quad (5.7)$$

which flattens magnitude and zeroes phase at once. Exact inversion is only safe for the minimum-phase part of the response; deep, non-minimum-phase room nulls cannot be inverted without enormous, ringing boosts [9, 8]. Four mechanisms regularise and shape C_0 into the correction that is actually applied.

1. Soft-knee boost ceiling. Rather than hard-clamping the boost (which leaves a kink at every notch), the *boost* portion of the correction magnitude (in dB) is bent towards a ceiling L (default +12 dB) with a tanh knee of width $\kappa = \min(6, L)$ dB:

$$C_{\text{dB}} \mapsto \begin{cases} C_{\text{dB}}, & C_{\text{dB}} \leq L - \kappa, \\ (L - \kappa) + \kappa \tanh \frac{C_{\text{dB}} - (L - \kappa)}{\kappa}, & C_{\text{dB}} > L - \kappa. \end{cases} \quad (5.8)$$

The boost passes untouched up to $L - \kappa$, then saturates smoothly and asymptotes to L . Cuts ($|C| < 1$) are left alone — attenuating a peak is always safe, only filling a null is dangerous. This is the practical form of regularised inversion discussed in [10].

2. Low-frequency slope. To stop the correction from boosting subsonic content where the speaker has no output and only noise, C is multiplied by a 2nd-order Butterworth **high-pass target** at $f_c = 30$ Hz,

$$H_{\text{hp}}(f) = \frac{(j\omega)^2}{(j\omega)^2 + \sqrt{2}(j\omega) + 1}, \quad \omega = f/f_c, \quad (5.9)$$

so below f_c the correction rolls off rather than chasing the woofer’s natural roll-off to infinity.

3. Correction level. A user control morphs continuously from bypass to full correction by raising the correction to a power in the log/phase domain,

$$C \mapsto |C|^\lambda e^{j\lambda \arg C}, \quad \lambda \in [0, 1], \quad (5.10)$$

so $\lambda = 0$ gives unity (no correction) and $\lambda = 1$ the full inverse; intermediate values apply a fractional correction (useful when full inversion sounds over-processed).

4. Analysis band. Outside a user range $[f_{\text{lo}}, f_{\text{hi}}]$ the correction is faded back to unity through a half-octave raised-cosine skirt $w(f)$,

$$C \mapsto (1 - w(f)) + w(f) C, \quad (5.11)$$

so out-of-band microphone/room noise and content the speaker cannot reproduce are neither inverted nor injected into the exported filter.

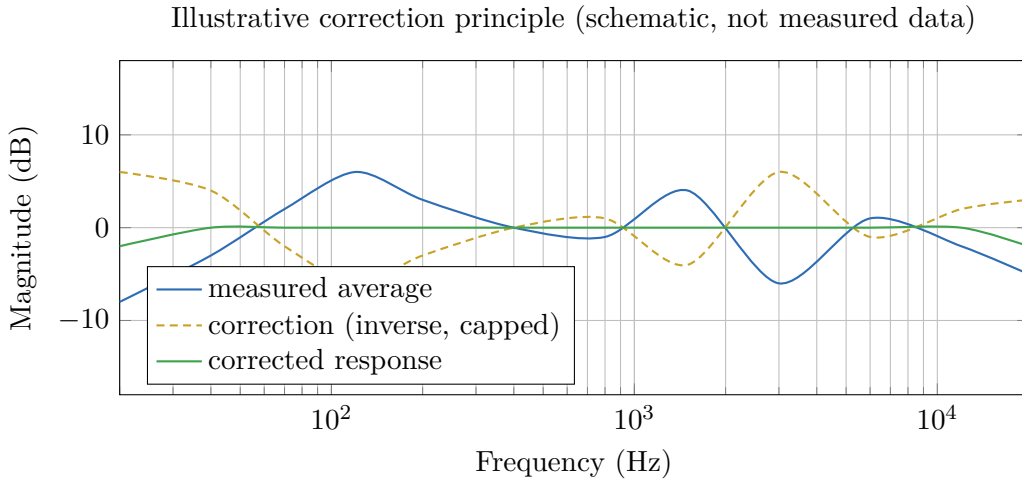


Figure 5.2: Schematic of the correction principle: the correction (gold, dashed) is the regularised inverse of the smoothed measured average (blue); applying it yields an approximately flat corrected response (green). The boost ceiling prevents the correction from fully inverting deep notches; the low-frequency slope rolls the correction off below the target corner. *This plot is illustrative, not real measurement data.*

5.5 Subwoofer phase alignment

Where a main and a subwoofer overlap around a crossover, their outputs add as *complex* pressures: a flat summed response requires the two to be in phase through the crossover region, otherwise the overlap shows a dip (or, if anti-phase, a deep cancellation) [12]. SuperMoTo can steer the corrected main’s phase onto the subwoofer’s so the pair sums coherently, without touching either magnitude.

5.5.1 Preserving the relative main/sub timing

A second measurement set (the subwoofer) is loaded, taken at the *same* positions as the main and paired by load order. The crucial step is that each sub measurement is **delay-anchored on the corresponding main measurement’s delay** rather than having its own delay estimated and removed. If each file were independently zeroed, the physically meaningful main-vs-sub arrival-time difference at each position would be discarded; anchoring keeps it, so the averaged sub response $\bar{S}(f)$ retains its true phase relative to $\bar{H}$.

5.5.2 The all-pass alignment term

From the (unit-magnitude) phase of the sub, $\hat{s}$ (its phasor $\bar{S}_s/|\bar{S}_s|$, optionally time-shifted as in Section 5.5.3), an **all-pass** factor is built and blended towards “no change” above the crossover,

$$A(f) = \frac{(1 - w_x) + w_x \hat{s}(f)}{|(1 - w_x) + w_x \hat{s}(f)|}, \quad C \mapsto C \cdot A(f), \quad (5.12)$$

where the alignment weight $w_x(f)$ is **1 at and below the crossover** and is released to 0 over one octave above it (raised cosine), so the main keeps its own flat-phase correction in its passband and only inherits the sub’s phase where the two actually overlap. The renormalisation makes A strictly phase-only — **the main’s magnitude correction is unchanged**. Blending the *phasors* (rather than scaling an unwrapped phase angle) keeps the rotation on the short arc between 1 and $\hat{s}$, so A applies only the minimal phase shift and never unwinds the sub’s large accumulated low-frequency phase. **Invert** negates $\bar{S}$ to flip the sub polarity if it is wired (or behaves) out of phase.

5.5.3 Bulk time alignment: the *Mains delay* control

The sub usually lags the mains by several milliseconds — its own group delay plus any extra path or processing latency. Folding that whole offset into the all-pass $A(f)$ would demand a long, low-frequency-heavy correction FIR. Instead the **Mains delay** control t_a (−40 to 40 ms) declares a bulk delay that the user applies *physically* to the main output(s) with the per-output **delays** in the matrix, leaving the all-pass to carry only the *residual*. The sub phasor entering the blend is advanced by t_a before its phase is taken,

$$\hat{s}(f) = \frac{\hat{S}(f)}{|\hat{S}(f)|}, \quad \hat{S}(f) = \pm \bar{S}_s(f) e^{j2\pi f t_a}, \quad (5.13)$$

so when t_a matches the measured offset the corrected main’s phase is already flat through the crossover and $A \rightarrow 1$: a short, well-behaved FIR. The corrected-phase curve visibly flattens as t_a is tuned to the right value.

On loading the sub set the plugin estimates the offset automatically, from a least-squares fit of the unwrapped, delay-anchored sub phase over $[f_c/2, 2f_c]$ — the local group delay $\hat{\tau} = -d\varphi/d\omega$ — and shows it as a recommendation. It is **not** applied automatically: t_a is a promise that the matching physical delay will be set in the matrix, so the user opts in by moving the slider, and the on-screen message then states the exact delay (and which output) to dial in. Positive t_a delays the main(s); negative delays the sub.

Applying it with the FIR (*Apply bulk delay*). To avoid the manual matrix step, tick **Apply bulk delay: Export correction IR** then writes t_a onto the assigned output's **delay** at the same time it loads the FIR, so the promised bulk delay and the residual-only correction land together. The value is clamped to the output's non-negative range, so a *negative* t_a (delay the sub instead) clamps to zero — in that case delay the subwoofer's output by $|t_a|$ by hand. This is the per-speaker measured route of Section 3.5.1: each main's exported delay equals its path-length term, so all mains (and the sub) arrive together at the centre.

5.5.4 The complete correction filter

Collecting the stages of Sections 5.4–5.5.4, the correction applied at each frequency bin is the single complex expression

$$C(f) = (1 - w(f)) + w(f) A(f) \mathcal{P}_\lambda \left[H_{\text{hp}}(f) \mathcal{B}[1/h(f)] \right] \quad (5.14)$$

evaluated on the Welch bin grid ($f_k = k f_s / W$). It is this spectrum $C(f)$ — not an analytic filter — that Section 5.6 resamples and inverse- transforms into the exported FIR. Reading (5.14) from the inside out, the operators are exactly the design steps above:

$$h(f) = \frac{\bar{H}_s(f)}{g_{\text{ref}}} \quad \text{normalised smoothed average (Section 5.4),} \quad (5.15)$$

$$\frac{1}{h(f)} = \frac{\overline{h(f)}}{|h(f)|^2} \quad \text{exact complex inverse, eq. (5.7),} \quad (5.16)$$

$$\mathcal{B}[z] = z 10^{(\ell(z_{\text{dB}}) - z_{\text{dB}})/20}, \quad z_{\text{dB}} = 20 \log_{10} |z| \quad \text{soft-knee boost limit (phase kept),} \quad (5.17)$$

$$\ell(g) = \begin{cases} g, & g \leq L - \kappa, \\ (L - \kappa) + \kappa \tanh \frac{g - (L - \kappa)}{\kappa}, & g > L - \kappa, \end{cases} \quad \kappa = \min(6, L), \quad L = \text{max boost (dB),} \quad (5.18)$$

$$H_{\text{hp}}(f) = \frac{(j\omega)^2}{(j\omega)^2 + \sqrt{2}(j\omega) + 1}, \quad \omega = \frac{f}{f_c} \quad \text{LF slope, } f_c = 30 \text{ Hz,} \quad (5.19)$$

$$\mathcal{P}_\lambda[z] = |z|^\lambda e^{j\lambda \arg z} \quad \text{correction level } \lambda \in [0, 1], \quad (5.20)$$

$$\hat{s}(f) = \frac{\pm \bar{S}_s(f) e^{j2\pi f t_a}}{|\bar{S}_s(f)|} \quad \text{time-aligned sub phasor, } t_a = \text{Mains delay,} \quad (5.21)$$

$$A(f) = \frac{(1 - w_x) + w_x \hat{s}(f)}{|(1 - w_x) + w_x \hat{s}(f)|} \quad \text{sub phase alignment } (A \equiv 1 \text{ if no sub),} \quad (5.22)$$

$$w(f) \in [0, 1] \quad \text{analysis-band weight (raised-cosine skirts).} \quad (5.23)$$

Two boundary cases complete the definition: where $|h(f)|$ is negligible the inverse is replaced by the boost ceiling (rather than diverging), and cuts ($z_{\text{dB}} \leq 0$) pass $\mathcal{B}$ unchanged since $L - \kappa \geq 0$ — only *boosts* are limited. Setting $\lambda = 1$, no subwoofer ($A \equiv 1$) and a fully open analysis band ($w \equiv 1$) reduces (5.14) to the boost-limited, LF-sloped exact inverse $H_{\text{hp}} \mathcal{B}[1/h]$.

5.6 Rendering the FIR filter

Both exports turn a complex spectrum into a finite impulse response on the FIR’s frequency grid ($N = \text{nextPow2}(\text{firLength})$ bins). The correction can be rendered in two ways, selected by the **Phase** control: **linear** (mixed-phase) or **minimum**. The measured-IR export is always linear-phase.

5.6.1 Linear- / mixed-phase rendering

The resampled spectrum is given a **half-length linear phase shift** — multiplying bin k by $e^{-j\pi k} = (-1)^k$ — before the inverse FFT. Because the correction is a *mixed-phase* filter (it deliberately carries phase and all-pass content, including the subwoofer alignment, so it is not causal on its own), centring its energy at sample $N/2$ is what makes it realisable as an ordinary FIR. A **Tukey window** (10 % taper) centred on $N/2$ then cleans up the edges. FIR design by frequency sampling and windowing is standard [4].

The cost of centring is a constant latency of $\approx N/2$ samples on that output. SuperMoTo’s per-output convolution uses the WDL zero-latency (uniformly partitioned) convolution engine [13], so the filter adds no *algorithmic* block latency beyond that fixed $N/2$ offset — which the matrix’s inter-output latency compensation (Chapter 3) then absorbs automatically so all outputs stay time-aligned.

5.6.2 Minimum-phase rendering

When low latency matters more than phase accuracy — typically **monitoring while tracking** — the correction can instead be rendered as a **minimum-phase** filter built from its *magnitude alone*. As defined in Section 1.2.5, a minimum-phase response is the unique causal, stable response whose log-magnitude and phase form a Hilbert-transform pair (eq. (1.2)), so it is fully determined by $|C(f)|$ [4]. SuperMoTo computes it by the real-cepstrum method: with $c[n] = \mathcal{F}^{-1}\{\log |C|\}$ the real cepstrum, the minimum-phase log-spectrum is recovered by keeping its causal part,

$$\hat{c}[n] = c[n] \ell[n], \quad \ell[n] = \begin{cases} 1, & n = 0, \ n = N/2, \\ 2, & 1 \leq n < N/2, \\ 0, & N/2 < n \leq N - 1, \end{cases} \quad (5.24)$$

and the impulse response is $\mathcal{F}^{-1}\{\exp \mathcal{F}\{\hat{c}\}\}$, tapered only at the tail (its energy is front-loaded near $n = 0$) [4]. The magnitude correction is *identical* to the linear-phase version; what is given up is the excess-phase flattening and, with a subwoofer, the all-pass alignment of Section 5.5.4 (a minimum-phase filter cannot impose an arbitrary phase). In return the filter is causal and front-loaded, so it adds **essentially no bulk latency**.

Because the inter-output latency compensation keys off each FIR’s measured peak position (Chapter 3), it recognises the two renderings automatically: a linear-phase IR reports $\approx N/2$, a minimum-phase IR reports ≈ 0 , and no special handling is needed. Note, however, that compensation aligns every fed output to the *longest* engaged FIR, so the low-latency benefit is only fully realised when *all* engaged outputs use minimum-phase (or no) FIRs — a single linear-phase FIR in the active configuration pulls the rest back up to its $N/2$ delay. The on-screen correction-phase plot always shows the linear-phase design target; in minimum-phase mode the exported filter matches the displayed *magnitude* but carries its minimum-phase companion instead.

5.7 Exporting

- **Export IR** — saves the *measured* response itself (the delay-aligned complex average), band-limited to the analysis range so out-of-band room/mic noise does not leak into the file.
- **Export correction IR** — renders the *correction* C (Section 5.4) to a mono 32 bit-float wav of a selectable FIR length and **Phase** type (Section 5.6), and can assign it directly to an output of the monitoring part. In *linear* phase the energy is centred at half the length, adding $\text{firLength}/2$ samples of delay on that output (absorbed automatically by the inter-output latency compensation); in *minimum* phase it is front-loaded and adds essentially none (Section 5.6.2).

Chapter 6

Workflows

6.1 A — Stereo monitoring with corrected speakers

1. Matrix view (or Config tool → Stereo 2.0 → Apply to A): route input 1→out 1, input 2→out 2.
2. Calibration, **Dry** mode: measure outputs 1 and 2, several mic positions around the listening spot.
3. Analysis: load each speaker's set, design the correction, **Export correction IR** and assign it to the output. Repeat per speaker.
4. Calibration, **FIR** mode: re-measure to verify the corrected response is flat.

6.2 B — 2.1 / bass-managed system with an aligned subwoofer

1. Config tool → Stereo 2.1 (or 5.1 / 7.1) → set the crossover → Apply.
2. Measure and correct each main (workflow A).
3. Measure the **sub** alone (Dry) at the same mic positions as one main.
4. Analysis: load the main set, then **Load sub measurements**; set the crossover (toggle **Invert** if the sub is wired out of polarity) — the corrected-main phase is steered onto the sub's through the crossover.
5. Read the recommended **Mains delay**; raise the slider until the corrected-main phase flattens through the crossover, then note the delay it asks for. Export and assign.
6. Time-align the drivers physically: enter that **Mains delay** in the matrix's per-output **delays**; SuperMoTo then automatically delays the sub output to match the mains' FIR latency (gold LED on the sub output). Verify in Calibration **System** mode by measuring the input: the summed response should be smooth through the crossover.

6.3 C — Verifying the complete system end to end

Calibration, **System** mode, select the plugin input your source uses. The stimulus runs through the engaged configuration exactly as monitored (routing, crossover, FIRs, latency compensation). Analysis of that capture shows the real in-room system response.

6.4 D — Multiple speaker sets / quick A/B

Store different rigs or rooms in configurations A–F (e.g. A = mains, B = mains+sub, C = nearfields) and switch from the top bar. Use **Exclusive** for A/B, non-exclusive to sum. Collapse (▲) to a compact output-strip-only window.

6.5 E — Ambisonics monitoring

1. Set the plugin bus large enough (e.g. 24 in/out for order 3: 16 B-format inputs, 16 speakers and a sub).
2. Config tool → **Ambisonics**, pick the **Order**; feed your B-format (AmbiX) onto inputs $1-(\text{order} + 1)^2$.
3. Enter each loudspeaker's azimuth/elevation and output channel (start from the default rig), keep bass management on, set the crossover, then **Apply** (Section 3.5.1).
4. Measure and correct each speaker (workflow A) and phase-align the sub (workflow B); the decode, corrections and latency compensation then run together as one engine.

Chapter 7

Building from source

SuperMoTo is CMake based and mirrors the MechanOdd project layout. JUCE and the FxmeJuceTools user module are expected as siblings of the repository:

```
../JUCE/                               juce-framework/JUCE checkout
../JUCE/usermodules/FxmeJuceTools/      symlink to FxmeJuceTools/module/FxmeJuceTools

git submodule update --init --recursive # FxmeFX + WDL
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

Produced formats: VST3, AU, Standalone (discrete in/out, selectable 8 / 16 / 24 / 32, default 8). Continuous-integration builds for Linux, macOS (universal) and Windows are provided by the GitHub Actions workflow `.github/workflows/release.yml`, triggered by pushing a version tag (`vX.Y.Z`).

7.1 License

LGPL-3.0-or-later — © 2023–2026 Olivier Doaré.

Appendix A

Glossary

Cross-references point to where each term is developed in the text.

All-pass / excess phase

A factor with *unit magnitude* that changes only phase (eq. (1.1)). It carries the “excess” phase a response has beyond the minimum-phase minimum — pure delay, reflections, out-of-phase summation. Not removable by a stable causal filter; SuperMoTo deliberately *adds* a short all-pass to steer a main onto a subwoofer’s phase (Section 5.5.3).

Cepstrum (real)

The inverse Fourier transform of the log-magnitude spectrum, $c[n] = \mathcal{F}^{-1}\{\log |C|\}$. Keeping its causal part reconstructs the minimum-phase response used for low-latency rendering (Section 5.6.2).

Coherence

The fraction of the measured output linearly explained by the input at each frequency (0 to 1); it quantifies how much a Welch estimate can be trusted (Section 5.1).

Comb filtering

Regularly spaced peaks and notches produced when the direct sound sums with a delayed copy (a reflection); wide-band and tied to geometry (Section 5).

Crossover

The frequency region where two drivers (e.g. main and subwoofer) hand over. A flat sum there is a phase/timing problem as much as a level one (Section 5.5.3).

FIR (finite impulse response) filter

A filter that convolves the signal with a finite list of coefficients (the impulse response). The exported correction is an FIR loaded per output; length N sets the frequency resolution f_s/N and, if linear-phase, the latency $N/2$.

Group delay

$-d(\arg H)/d\omega$: the delay a response imposes near each frequency. Used to auto-detect the main/sub offset (Section 5.5.3).

H_1 estimator

The cross-spectrum / input auto-spectrum ratio S_{xy}/S_{xx} averaged over Welch segments; the noise-robust transfer function SuperMoTo measures (Section 5.1).

Hilbert transform

The operator linking log-magnitude and phase of a minimum-phase response (eq. (1.2)); lets phase be recovered from magnitude (Section 1.2.5).

Impulse response (IR)

A system's output to a unit impulse; the time-domain twin of the transfer function. Exported as a wav for the FIR engine and the measured-response file (Section 5.6).

Inter-output latency compensation

Automatic per-output delay so every engaged output stays time-aligned with the longest FIR's bulk latency (Chapter 3); shown by a gold LED on the output.

Linear-phase

A filter whose phase is exactly linear in frequency, i.e. a pure delay across the band; it can impose an arbitrary phase correction but adds $N/2$ samples of latency (Section 5.6).

Minimum-phase

A causal, stable response whose inverse is also causal and stable (all poles and zeros inside the unit circle) — the only kind that is fully invertible by a realisable filter. Defined physically and mathematically in Section 1.2.5.

Polarity (Invert)

A frequency-independent 180° sign flip, as opposed to a delay (which is frequency-dependent). **Invert** flips a subwoofer wired out of polarity (Section 5.5.3).

Regularization

Bounding the inverse where the measured magnitude is tiny, so deep notches do not demand infinite boost; here a soft-knee (**tanh**) ceiling on the correction gain (Section 5.4).

Room mode

A standing-wave resonance of the room producing large, strongly position-dependent peaks and dips at low frequency, with audible ringing (Section 5).

SBIR

Speaker-boundary interference response: the cancellation dip in the upper bass set by the speaker-to-wall distance (Section 5).

Smoothing (fractional-octave)

Averaging the complex response over a proportional bandwidth (e.g. 1/6 octave), finer in the bass and coarser in the treble, so the correction follows trends rather than single-seat interference (Chapter 5).

Spatial averaging

Averaging the (delay-aligned) responses of several microphone positions, so the correction targets what is consistent across the listening area rather than one seat (Section 5.1.4).

Transfer function

The complex ratio $H(f) = Y(f)/X(f)$ of output to input spectrum — magnitude and phase together — that describes a linear system (Section 5.1).

Welch method

Power/cross-spectra estimated by averaging windowed, overlapping FFT segments; trades resolution for variance to give a usable in-room transfer function (Section 5.1).

Appendix B

Ambisonics primer

This appendix gives the background behind the Ambisonics decoders the configuration tool generates (Section 3.5.1): the spherical-harmonic representation, the orders, the AmbiX (ACN/SN3D) convention, how a decoder is computed, and the $\max\text{-}r_E$ weighting.

B.1 The idea: encode the field, decode to the rig

Ambisonics represents a *sound field* around a central point as a set of **spherical-harmonic (SH) components**, independently of any loudspeaker layout [14, 15]. A unit-amplitude plane wave arriving from direction $\Omega = (\theta, \varphi)$ (azimuth θ , elevation φ) carrying signal s is **encoded** into channels

$$B_j = Y_j(\Omega) s, \quad (\text{B.1})$$

where the Y_j are the spherical harmonics; a whole field is the superposition over all incident directions. Because the encoding does not mention speakers, one B-format signal can be **decoded** to any rig. Reproduction truncates the expansion at a finite **order** N , which sets the angular resolution and the size of the region around the centre where the field is faithfully reconstructed.

B.2 Spherical harmonics

The real spherical harmonics Y_n^m have a **degree** $n \geq 0$ and an **order** $m \in [-n, n]$; they form an orthogonal basis on the sphere, and degree n carries the angular detail of that order. Writing the unit direction as

$$x = \cos \varphi \cos \theta, \quad y = \cos \varphi \sin \theta, \quad z = \sin \varphi, \quad (\text{B.2})$$

the SN3D-normalised real harmonics used by SuperMoTo are, up to third degree,

$$\begin{aligned} n = 0 : Y_0 &= 1, \\ n = 1 : Y_1 &= y, \quad Y_2 = z, \quad Y_3 = x, \\ n = 2 : Y_4 &= \sqrt{3}xy, \quad Y_5 = \sqrt{3}yz, \quad Y_6 = \frac{1}{2}(3z^2 - 1), \quad Y_7 = \sqrt{3}xz, \quad Y_8 = \frac{\sqrt{3}}{2}(x^2 - y^2), \\ n = 3 : Y_9 &= \sqrt{\frac{5}{8}}y(3x^2 - y^2), \quad Y_{10} = \sqrt{15}xyz, \quad Y_{11} = \sqrt{\frac{3}{8}}y(5z^2 - 1), \quad Y_{12} = \frac{1}{2}z(5z^2 - 3), \\ Y_{13} &= \sqrt{\frac{3}{8}}x(5z^2 - 1), \quad Y_{14} = \frac{\sqrt{15}}{2}z(x^2 - y^2), \quad Y_{15} = \sqrt{\frac{5}{8}}x(x^2 - 3y^2). \end{aligned}$$

Each is scaled so its maximum magnitude over the sphere is 1 (the SN3D convention, below).

B.3 Ambisonic order

An order- N signal keeps every degree $n \leq N$. In 3D (**periphonic**) that is

$$K = (N + 1)^2 \quad \text{channels}, \quad (\text{B.3})$$

so orders 1/2/3 use 4/9/16 channels and need at least that many loudspeakers to decode. A horizontal-only (**pantophonic**) signal keeps just $m = \pm n$, giving $2N + 1$ channels. Higher order narrows the directional “beam” of each harmonic, improving localisation and enlarging the usable listening area, at the cost of more channels and speakers (Table B.1).

Order N	3D channels $(N + 1)^2$	2D channels $2N + 1$	default rig in the tool
1	4	3	8 (cube)
2	9	5	12 (icosahedron)
3	16	7	16 (dome)

Table B.1: Ambisonic order vs. channel count and the configuration tool’s default periphonic rig.

B.4 The AmbiX convention: ACN and SN3D

Two choices must be fixed to exchange B-format: the channel *ordering* and the *normalisation*. The modern standard is **AmbiX** [16]:

- **ACN** (Ambisonic Channel Number) orders the channels by the single index

$$j = n^2 + n + m, \quad (\text{B.4})$$

so $j = 0, 1, 2, 3, \dots$ are $W, Y, Z, X, \dots$ — note Y ($m = -1$) before Z ($m = 0$) before X ($m = +1$).

- **SN3D** (Schmidt semi-normalisation) scales each harmonic so its peak magnitude is 1; it relates to the fully orthonormal N3D form by $Y_n^{\text{SN3D}} = Y_n^{\text{N3D}} / \sqrt{2n + 1}$.

This supersedes the legacy **FuMa** (Furse-Malham) ordering $W, X, Y, Z, \dots$ with its $1/\sqrt{2}$ weighting on W , which was limited to third order. SuperMoTo expects AmbiX on the first K plugin inputs.

B.5 Decoding

A **decoder** maps the K B-format channels to L loudspeaker feeds. Collect the speaker directions in the $K \times L$ *re-encoding* matrix $\mathbf{C}$ with $C_{j,s} = Y_j(\Omega_s)$: driving the speakers with gains $\mathbf{g}$ recreates the components $\mathbf{Cg}$. A good decode $\mathbf{D}$ ($\mathbf{g} = \mathbf{D}\mathbf{b}$) makes that match the input $\mathbf{b}$, i.e. $\mathbf{CD} \approx \mathbf{I}$. The general solution is the **mode-matching** pseudo-inverse $\mathbf{D} = \mathbf{C}^+$. For a *near-uniform* layout the harmonics stay orthogonal when sampled at the speakers, $\mathbf{CC}^T \approx \frac{L}{K} \mathbf{I}$, and the pseudo-inverse collapses to the scaled transpose — the **sampling (projection) decoder**

$$D_{s,j} = \frac{1}{L} Y_j(\Omega_s), \quad (\text{B.5})$$

which is what the tool builds. It is exact for ideal layouts and degrades gracefully for mildly irregular ones; strongly irregular rigs are better served by an explicit pseudo-inverse or AllRAD decoder, which is why the tool also lets you trim each speaker’s gain/angle/radius by hand.

B.6 The max- r_E weighting

At the centre, localisation is governed by two cues: the **velocity vector** $\mathbf{r}_V$ (low frequency, from the summed particle velocity) and the **energy vector** $\mathbf{r}_E$ (above ~ 700 Hz, from the summed

energy) [15, 17]. The basic decoder of (B.5) makes $|\mathbf{r}_V| = 1$ (correct low-frequency cue) but spreads energy, so $|\mathbf{r}_E| < 1$. The **max- r_E** decoder applies a per-degree gain a_n to maximise $|\mathbf{r}_E|$,

$$D_{s,j} = \frac{1}{L} a_{n(j)} Y_j(\Omega_s), \quad (\text{B.6})$$

where the optimal weights are samples of the Legendre polynomials,

$$a_n = P_n(r_E^{\max}), \quad (\text{B.7})$$

with $r_E^{\max}$ the largest root of $P_{N+1}(x)$ (the order- N energy limit). Normalised to $a_0 = 1$, the values used by SuperMoTo are listed in Table B.2. Max- r_E slightly widens phantom sources but markedly improves directional stability and timbre across the listening area, so it is the default for monitoring.

Order N	$r_E^{\max}$	a_0	a_1	a_2	a_3
1	0.5774	1	0.5774	–	–
2	0.7746	1	0.7746	0.4000	–
3	0.8611	1	0.8611	0.6123	0.3047

Table B.2: max- r_E per-degree gains (normalised $a_0 = 1$), from the largest root of P_{N+1} .

In SuperMoTo the resulting matrix is finally scaled so an on-axis plane wave decodes to about unity at the nearest speaker, written as harmonic $\rightarrow$ speaker frames, and combined with the radius compensation and bass management of Section 3.5.1. For deeper treatments see Zotter & Frank [17] and Daniel [15].

Bibliography

- [1] P. D. Welch, “The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms,” *IEEE Trans. Audio Electroacoust.*, vol. 15, no. 2, pp. 70–73, 1967.
- [2] J. S. Bendat and A. G. Piersol, *Random Data: Analysis and Measurement Procedures*, 4th ed. Hoboken, NJ: Wiley, 2010.
- [3] K. Shin and J. K. Hammond, *Fundamentals of Signal Processing for Sound and Vibration Engineers*. Chichester: Wiley, 2008. (Frequency-response estimators H_1/H_2 and the coherence function.)
- [4] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [5] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, 1978.
- [6] S. Müller and P. Massarani, “Transfer-function measurement with sweeps,” *J. Audio Eng. Soc.*, vol. 49, no. 6, pp. 443–471, 2001.
- [7] P. D. Hatziantoniou and J. N. Mourjopoulos, “Generalized fractional-octave smoothing of audio and acoustic responses,” *J. Audio Eng. Soc.*, vol. 48, no. 4, pp. 259–280, 2000.
- [8] J. N. Mourjopoulos, “Digital equalization of room acoustics,” *J. Audio Eng. Soc.*, vol. 42, no. 11, pp. 884–900, 1994.
- [9] S. T. Neely and J. B. Allen, “Invertibility of a room impulse response,” *J. Acoust. Soc. Am.*, vol. 66, no. 1, pp. 165–169, 1979.
- [10] O. Kirkeby and P. A. Nelson, “Digital filter design for inversion problems in sound reproduction,” *J. Audio Eng. Soc.*, vol. 47, no. 7/8, pp. 583–595, 1999.
- [11] F. E. Toole, *Sound Reproduction: The Acoustics and Psychoacoustics of Loudspeakers and Rooms*, 3rd ed. New York: Routledge, 2017. (Spatial averaging and the case against single-point room correction.)
- [12] S. H. Linkwitz, “Active crossover networks for noncoincident drivers,” *J. Audio Eng. Soc.*, vol. 24, no. 1, pp. 2–8, 1976.
- [13] W. G. Gardner, “Efficient convolution without input-output delay,” *J. Audio Eng. Soc.*, vol. 43, no. 3, pp. 127–136, 1995.
- [14] M. A. Gerzon, “Periphony: With-height sound reproduction,” *J. Audio Eng. Soc.*, vol. 21, no. 1, pp. 2–10, 1973.
- [15] J. Daniel, *Représentation de champs acoustiques, application à la transmission et à la reproduction de scènes sonores complexes dans un contexte multimédia*. PhD thesis, Université Paris 6, 2000. (Ambisonic encoding/decoding, r_V/r_E and $\max\text{-}r_E$.)

- [16] C. Nachbar, F. Zotter, E. Deleflie, and A. Sontacchi, “AmbiX — a suggested ambisonics format,” in *Proc. 3rd Ambisonics Symposium*, Lexington, 2011.
- [17] F. Zotter and M. Frank, *Ambisonics: A Practical 3D Audio Theory for Recording, Studio Production, Sound Reinforcement, and Virtual Reality*. Cham: Springer, 2019 (open access).